

PrepBench: A Factorized Benchmark for Cost-Aware Data-Preparation Pipeline Search [Experiment, Analysis & Benchmark]

Jing Chang
Shenzhen University
2350273007@email.szu.edu.cn

Chang Liu
Shenzhen University
2300271084@email.szu.edu.cn

Jinbin Huang
Shenzhen University
jbhuang@szu.edu.cn

Yu-Xuan Qiu
Beijing Institute of Technology,
Zhuhai
qiu@bit.edu.cn

Rui Mao
Shenzhen University
mao@szu.edu.cn

Jianbin Qin*
Shenzhen University
qinjianbin@szu.edu.cn

ABSTRACT

Automated data-preparation systems often bundle rich pipeline structures, cost models, staged evaluation, and budget-aware pruning inside one end-to-end optimizer. This makes it hard to attribute gains to individual design choices or to distinguish accuracy improvements from changes in resource allocation. We present PrepBench, a factorized benchmark for cost-aware preprocessing-pipeline search. PrepBench varies common design axes in a shared execution substrate while holding the operator library, estimator family, metric, data splits, and wall-clock budgets fixed.

Across 56 OpenML tabular classification tasks, five seeds, and 60 s/300 s/600 s budgets, we compare nine controlled configurations using paired tests, equivalence tests, allocation metrics, and candidate-level provenance traces. The main finding is diagnostic: within this substrate and budget regime, random feasible pipelines are strong baselines. At the primary 300 s and 600 s budgets, the random controls significantly outrank the guided structured variants, while critical-path costing, staged evaluation, and budget-aware pruning are statistically equivalent to a plain DAG baseline in final balanced accuracy.

PrepBench provides a reusable benchmark and measurement protocol for preparation-pipeline search: random feasible controls, equivalence tests, candidate provenance, and cost-model calibration are needed to interpret structured and cost-aware search claims.

PVLDB Reference Format:

Jing Chang, Chang Liu, Jinbin Huang, Yu-Xuan Qiu, Rui Mao, and Jianbin Qin. PrepBench: A Factorized Benchmark for Cost-Aware Data-Preparation Pipeline Search [Experiment, Analysis & Benchmark]. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/prepbench2027/prepbench>.

*Corresponding author(s)

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Data preparation transforms raw tabular data into a form suitable for downstream analytics through imputation, type-aware encoding, scaling, feature selection, and resampling. It is widely regarded as one of the most time-consuming stages of the analytics pipeline [5]. To relieve this burden, the database community has built systems that automate preparation, and these systems have grown steadily more elaborate: from interactive cleaning aligned with a downstream model [17], to task sequencing for web data [2], to differentiable search over preprocessing pipelines [19]. This trajectory raises a measurement question that is rarely isolated: when does richer preparation *structure*, together with the cost-aware machinery needed to search it, earn its added complexity? A cautionary data point comes from CleanML [20], which showed that the effect of a cleaning choice on downstream accuracy is frequently small or statistically insignificant and must be established with hypothesis testing rather than asserted. If the payoff of *cleaning* can be marginal, the payoff of elaborate *search over* preparation pipelines deserves the same scrutiny.

Concretely, automating data preparation is a search problem: given a budget, which operators to apply, in what order, and over which columns. Recent systems attack it with four interacting mechanisms. Directed acyclic graph (DAG) [25] representations express branch-and-merge preparation that processes numerical and categorical columns differently before merging. Execution-cost models penalize expensive preparation plans. Staged evaluation screens candidate pipelines on a cheap subsample before committing to full validation. Budget-aware pruning discards plans that cannot complete within the remaining wall-clock budget. Each is individually plausible, but they are almost always bundled together inside one proposed method. As a result, when such a method reports a gain, it is unclear whether the credit belongs to preparation structure, to the cost model, to pruning, to the evaluation policy, or simply to a different search space or a more favorable budget.

Rather than proposing yet another preparation-search optimizer and claiming it is best, this paper asks a more basic question: *when does structure-aware, cost-aware preparation search actually help under wall-clock budgets, and when is it indistinguishable from far simpler alternatives?* Answering this requires holding everything else fixed—the operator library, the datasets, the budget, and the way candidates are scored—and changing only the mechanism

under test. A single proposed system cannot provide that isolation, because adopting it changes many factors at once. We therefore build a controlled benchmark that factorizes the design space and varies one axis at a time within a shared execution substrate.

This framing is also why the benchmark is a data-management contribution, not only an AutoML comparison. A preparation pipeline is a data-processing workflow: operators transform typed columns, branches introduce dependencies, and budgeted search allocates work across many candidate workflows. The systems questions are therefore familiar to database research: how should the workflow space be represented, how should execution cost be estimated, which candidates should be evaluated under a budget, and how should provenance establish that a reported result is supported by completed work? PrepBench exposes these workflow, cost, and provenance dimensions explicitly.

We instantiate this as nine configurations in one substrate, varying four axes: preparation structure (linear chain, fixed type-split branch, typed DAG), execution-cost model (serial-sum vs. critical-path), evaluation policy (full vs. staged), and budget policy (agnostic vs. budget-aware pruning). Crucially, two random controls: random feasible DAG pipelines and flat random preparation, are included as first-class baselines, not sanity checks. The matrix is run on 56 OpenML tabular tasks, five seeds, and three wall-clock budgets (60s, 300s, 600s), and every comparative claim is backed by a hypothesis-testing protocol rather than asserted from mean scores.

The results caution against a simple “more preparation structure is better” story. Within this fixed substrate and budget regime, random feasible preparation pipelines and flat random preparation are strong accuracy controls, and the separation from the guided cluster is statistically significant under a Friedman–Nemenyi analysis [7, 32]. The combined DAG, critical-path, staged, budget-aware configuration substantially changes how the budget is spent: it screens and prunes many candidates and performs far fewer full evaluations, but it does not win, or even significantly change, final balanced accuracy. Finally, most budget value is realized before 300s; from 300s to 600s the additional accuracy is statistically detectable but practically negligible despite hundreds of extra full evaluations.

Table 1 situates PrepBench against prior evaluations of preparation-pipeline search and cleaning-for-ML. The table is not a scorecard of prior systems; many of those systems were designed to introduce new search methods rather than to serve as factorized benchmarks. Instead, it identifies evaluation dimensions that are rarely reported together: random feasible controls, equivalence testing for “no difference” claims, and candidate-level trace audits for score provenance.

Contributions. This paper makes three contributions.

- **A factorized benchmark for preparation-pipeline search.** We introduce PrepBench, a controlled benchmark that separates four design axes that are usually bundled together: preparation structure, execution-cost model, staged evaluation, and budget-aware pruning. The benchmark runs all configurations in one execution substrate with the same operator library, estimator family, data splits, metric, and wall-clock budgets.
- **A measurement protocol for trustworthy results.** PrepBench treats random feasible preparation pipelines as first-class controls, combines standard multi-dataset significance testing with TOST equivalence tests, and records candidate-level provenance traces. This lets us distinguish “no detected difference” from statistical equivalence, and lets us verify whether reported scores come from completed full evaluations or from fallback paths.
- **A diagnostic empirical finding about search structure.** Across 56 OpenML tasks [14, 28], five seeds, and three wall-clock budgets, random feasible preparation pipelines are stronger baselines than expected: at 300s and 600s they significantly outrank all guided and hand-designed structured variants while using fewer full evaluations. In contrast, critical-path costing, staged evaluation, and budget-aware pruning are statistically equivalent in final balanced accuracy to the plain DAG baseline, even though they substantially change evaluation allocation. Sensitivity runs show that this finding is not caused by the cost/complexity terms in the EA survival objective, and that stronger guided search is needed merely to reach the random-control score range.

Research questions. The study is organized around four top-level research questions, each answered in Section 5.

- **RQ1: Search-space structure.** Under a fixed operator library, estimator family, metric, and budget, do richer preparation structures—from linear chains to type-split branches and typed DAGs—improve final balanced accuracy over random feasible controls?
- **RQ2: Cost-aware allocation.** Do critical-path costing, staged evaluation, and budget-aware pruning improve final balanced accuracy, or do they mainly change how evaluation effort is allocated across full evaluations, screened candidates, and pruned candidates?
- **RQ3: Budget scaling and mechanisms.** How much additional value is obtained when the budget increases from 60 s to 300 s and from 300 s to 600 s, and what do incumbent traces, screening diagnostics, and cost-calibration plots reveal about the mechanisms behind the observed trends?
- **RQ4: Robustness and external context.** Do the conclusions survive provenance audits and completed-evaluation-only analysis, and how should a recent external end-to-end preparation-search system be interpreted as a contextual reference rather than as a factorized ablation?

2 BACKGROUND AND RELATED WORK

2.1 Automated Data Preparation and Cleaning for ML

Data cleaning and preparation have long been studied as data-management problems [6, 8, 22]. Interactive and statistically aware cleaning systems align repairs with a downstream model [17], while surveys emphasize the persistent challenges of data cleaning at scale [5]. A related line of work studies how cleaning choices affect machine-learning outcomes. CleanML is particularly relevant to our study: it evaluates the impact of data cleaning on classification accuracy using controlled randomness and multiple-hypothesis-testing corrections [20]. Our study moves one level up. Rather than asking whether a fixed cleaning operation improves a model, we ask whether searching over preparation-pipeline structure under a

Table 1: Representative automated data-preparation and AutoML systems, organized by the design axes that this paper factorizes and by the evaluation dimensions explicitly reported in their introducing papers. Capability columns describe whether the system includes a mechanism of that type. Evaluation columns describe what was reported in the corresponding study: a random feasible preparation control, multi-dataset significance testing, formal equivalence testing, and candidate-level provenance traces. ✓ means reported/present, ✗ means not reported or absent in that paper, and (∼) means partial. The table is not a limitation scorecard; PrepBench does not propose a new optimizer, but provides a controlled substrate and measurement protocol for isolating these axes.

System	Method	Capability axes				Reported evaluation dimensions			
		Struct.	Cost	Stage	Prune	Rand.	Stat.	Equiv.	Trace
<i>Data-preparation pipeline systems</i>									
Learn2Clean [2] (WWW’19)	RL	✓	✗	✗	✗	✗	✗	✗	✗
DeepLine [14] (KDD’20)	RL	✓	✗	✗	(~) [‡]	✗	✓	✗	✗
HAIPipe [4] (SIGMOD’23)	RL+human	✓	✗	✗	✗	✗	✓	✗	✗
DiffPrep [19] (SIGMOD’23)	diff.	✓	✗	✗	✗	(~)	✗	✗	✗
SAGA [24] (SIGMOD’23)	genetic	✓	✗	✗	(~) [‡]	✗	✗	✗	✗
CleanML [20] (ICDE’21)	—	✗	✗	✗	✗	✗	✓	✗	✗
CtxPipe [12] (SIGMOD’24)	RL+ctx	*	✗	✗	✗	✓	✓	✗	✗
<i>General AutoML (restricted prep. search), for reference</i>									
Auto-sklearn [11] (NeurIPS’15)	BO	(~)	✗	✗	✗	✗	✓	✗	✗
TPOT [23] (GECCO’16)	genetic	✓	✗	✗	✗	(~)	✓	✗	✗
PrepBench (Ours)	benchmark	✓	✓	✓	✓	✓	✓	✓	✓

[‡]SAGA prunes by *monotonicity*, not by a wall-clock budget—a different mechanism from PrepBench’s budget-aware pruning. [‡]DeepLine’s hierarchical action filtering prunes the RL agent’s *action space* for tractability, not candidate evaluations under a budget. *CtxPipe is described as a linear pipeline that generalizes to DAGs. Auto-sklearn searches a *fixed/semi-fixed* pipeline template (slots for data preprocessing, feature preprocessing, and a model), hence *Struct.* ∼. DiffPrep and TPOT each include a random-search variant in their own ablations (hence *Rand.* ∼); only PrepBench reports a random feasible pipeline as a first-class baseline together with an equivalence test and a candidate-level trace audit. ✗ means “not reported,” not “impossible.”

wall-clock budget improves the final result, and how such claims should be measured.

Recent systems automate the construction of preparation pipelines using a variety of search strategies. Learn2Clean sequences preparation tasks for web data [2]; DeepLine uses reinforcement learning with hierarchical action filtering [14]; HAIPipe combines machine- and human-generated pipelines [4]; SAGA applies genetic search and monotonicity-based pruning to cleaning pipelines [24]; DiffPrep relaxes discrete preprocessing choices into a differentiable search space [19]; and CtxPipe [12] uses data context to guide reinforcement-learning search. These systems motivate the design axes studied in this paper: pipeline structure, resource allocation, pruning, and evaluation policy. However, because these mechanisms are usually introduced together inside one optimizer, prior whole-system evaluations cannot easily determine which mechanism is responsible for an observed gain or loss. PrepBench therefore fixes the substrate and varies one axis at a time.

2.2 AutoML and Pipeline Search

Classic AutoML systems formulate model and pipeline construction as combined algorithm selection and hyperparameter optimization (CASH). Representative systems include Auto-WEKA, Auto-sklearn, TPOT, GAMA, FLAML, and AutoGluon [9–11, 13, 16, 23, 26, 29]. These systems are important reference points because they show how preprocessing, model selection, hyperparameter search [1], ensembling, and stopping policies interact in practical AutoML workflows. At the same time, their package-level comparisons are not designed to isolate preparation-specific mechanisms. A difference between two AutoML systems may come from the model

library, the search algorithm, the preprocessing space, ensembling, time accounting, or implementation overhead.

Our goal is therefore different from an AutoML leaderboard. We use external AutoML systems only as contextual references. The load-bearing comparisons in PrepBench are within-substrate comparisons in which the operator library, estimator family, metric, timeout policy, and dataset split are held fixed while preparation structure, cost model, staged evaluation, and budget-aware pruning are varied independently.

2.3 Budgeted, Multi-Fidelity, and Cost-Aware Search

Budgeted search and multi-fidelity optimization study how to allocate limited evaluation resources. Successive halving and Hyperband allocate partial budgets to many candidates and progressively promote promising ones [18]. Cost-aware bandit and budgeted-optimization methods similarly motivate reward-per-cost criteria and pruning under a limited resource budget. In AutoML, these ideas appear in systems that adaptively allocate time across configurations, estimators, or fidelities.

PrepBench adopts this perspective for data-preparation search, where candidate pipelines have heterogeneous evaluation costs and where cheap early screening may or may not predict full-validation quality. The benchmark therefore reports not only final balanced accuracy, but also full-evaluation counts, screened candidates, pruned candidates, promotion behavior, incumbent updates, and time-to-best. This makes it possible to distinguish an accuracy improvement from a change in resource allocation.

2.4 Workflow Cost, Provenance, and Systems Measurement

Preparation pipelines are data-processing workflows. A linear pipeline, a type-split branch, and a typed DAG can have the same operator library but different dependency structure, execution cost, and opportunities for parallelism. This connects preparation search to workflow optimization and scheduling, where notions such as makespan, critical path, and heterogeneous execution cost are central [27]. In our setting, however, the workflow is searched jointly with validation feedback: a candidate’s value depends on both its predictive score and the budget it consumes.

2.5 Benchmarking Methodology and Factorized Evaluation

Comparing learning systems across datasets has an established statistical methodology, including Friedman tests, post-hoc rank analysis, paired tests, and multiplicity control [7, 15]. Data-management benchmarks such as CleanML show the importance of applying such discipline to data quality and preparation studies [20]. We follow this practice and add two elements that are especially important for preparation search: equivalence testing and trace-level provenance auditing. Equivalence testing lets us substantiate a “no practical difference” claim rather than merely failing to reject a null hypothesis; provenance auditing verifies that reported run-level scores are supported by candidate-level evidence.

Table 1 summarizes how representative systems report the design and evaluation dimensions that PrepBench factorizes. The table is not intended to mark missing entries as limitations of prior systems; many of those systems were designed to introduce new optimizers, not to serve as attribution-oriented benchmarks. Instead, the table identifies the additional measurement dimensions needed for a study of data-preparation search: random feasible controls, factorized design axes, budget-sensitive allocation metrics, equivalence testing, and candidate-level trace provenance.

3 BENCHMARK DESIGN

3.1 Controlled Configuration Matrix

Table 2 shows the nine configurations. Each identifier encodes four design axes. The first field is preparation structure: L for a linear chain, B for a fixed type-split branch, and D for a typed DAG. The second field is the execution-cost model: S for serial-sum and C for critical-path. The third field is the evaluation policy: F for full evaluation and T for staged evaluation. The fourth field is the budget policy: A for budget-agnostic and B for budget-aware pruning.

The random controls are intentionally first-class. D-Rand samples feasible DAG preparation pipelines without guided selection, while Flat-Rand samples flat preparation pipelines. These controls test whether guided structure-aware search beats broad random exploration of the same or a simpler preparation space. To make this comparison interpretable, the benchmark uses the same operator library, estimator family, scoring function, train-test split, timeout accounting, and failure policy across all configurations; only the axes named in Table 2 change.

Table 2: Controlled configuration matrix. All configurations run in the same substrate and differ only by the listed axes. D-Rand and Flat-Rand are random preparation controls.

ID	Preparation structure	Cost model	Evaluation policy	Budget policy
L-S-F-A	Linear chain	Serial-sum	Full	Agnostic
B-S-F-A	Fixed type-split branch	Serial-sum	Full	Agnostic
D-S-F-A	Typed DAG	Serial-sum	Full	Agnostic
D-C-F-A	Typed DAG	Critical-path	Full	Agnostic
D-S-F-B	Typed DAG	Serial-sum	Full	Budget-aware
D-S-T-A	Typed DAG	Serial-sum	Staged	Agnostic
D-C-T-B	Typed DAG	Critical-path	Staged	Budget-aware
D-Rand	Random feasible DAG	Serial-sum	Full	Agnostic
Flat-Rand	Flat random pipeline	Serial-sum	Full	Agnostic

3.2 Preparation Substrate

Each candidate pipeline consists of data-preparation operators followed by an estimator. The operator library contains common tabular preparation steps: imputation, scaling, type-aware categorical encoding, feature selection, and resampling [3, 31]. DAG pipelines are typed and bounded by limits on the number of preparation nodes, depth, and branch count; the typed DAG can process numerical and categorical columns on separate branches before merging. Fixed-branch pipelines use a constrained numerical/categorical split. Linear pipelines restrict preparation to a single chain.

The critical-path cost model estimates the runtime of a branched preparation plan [21] as the longest branch path plus merge and synchronization terms; the serial-sum model sums all operator costs. Staged evaluation first evaluates candidate pipelines on a sub-sample and promotes candidates for full validation based on score and structural novelty. Budget-aware pruning discards candidate preparation plans whose estimated completion cost exceeds the remaining search budget.

Table 3: Operator library and estimator pool. The number of variants per family bounds the size of the search space the random controls must cover; “Scope” marks whether an operator may apply per-column-type (numerical/categorical) inside a typed DAG branch.

Family	Operators (variants)	#Var.	Scope
Imputation	mean / median / most-freq. / KNN	4	num+cat
Scaling	standard / min-max / robust	3	num
Cat. encoding	one-hot / ordinal / target	3	cat
Feature selection	variance / k-best	2	num
Feature engineer	polynomial feat. / tree embedding / PCA	3	num
Estimator	RF, GradBoost, LogReg, RBF-SVM	4	—

3.3 Search Procedure

All non-random configurations share one search procedure: a parallel-aware evolutionary algorithm (EA) over typed workflow graphs. A genome encodes the full pipeline—per-scope preprocessing branches, shared tail operators, and the downstream *estimator gene* (random

forest, gradient boosting, logistic regression, or RBF SVM with their hyperparameters); preparation structure and model are thus searched jointly. The population ($P=10$) is initialized by the same feasibility-constrained sampler used by the random controls (Section 3.4), so all configurations draw from one space. Each generation, parents are chosen by tournament selection (size 3); offspring are produced by mutation ($p=0.9$: add/remove a preprocessing node, replace an operator, mutate hyperparameters, or—only at the free-DAG structure level— split a branch) and branch/tail/estimator crossover ($p=0.7$). Offspring that violate the typing, depth, or branch-count constraints are repaired by reverting to the parent. Candidates are scored by balanced accuracy under the evaluation policy of the configuration (full 5-fold or stage-1 subsample), and survival is $(\mu+\lambda)$ truncation to P by fitness $f = \text{score} - 10^{-4}\hat{c} - 10^{-3}|\text{ops}|$, which lightly penalizes estimated cost $\hat{c}$ and pipeline complexity; the reported pipeline is the best-scoring incumbent. Because this survival objective could in principle bias the EA against larger preparation structures, we report a zero-penalty sensitivity in Section 5.2. Search stops at the wall-clock budget. The random controls (D-Rand, Flat-Rand) disable the evolutionary operators and instead draw feasible pipelines i.i.d. from the same generator, retaining the best by validation score.

3.4 Workflow Identity and Random-Control Fairness

A central risk in this benchmark is giving the random controls an easier or larger search space than the guided configurations. We therefore separate two notions of fairness. *Library fairness* requires every configuration to draw operators from the same preparation library and to evaluate candidates with the same downstream estimator and metric. *Space fairness* requires the random DAG control to sample only candidates that satisfy the same typing, depth, branch, and feasibility constraints as the guided typed-DAG configurations. Flat random preparation is intentionally simpler and is reported as a separate reference point rather than as an equal-space ablation.

We also audit this fairness empirically from the candidate traces. For each configuration and budget, the released summaries record pipeline size, depth, branch count, evaluation status, timeout/failure status, and fallback scores. These checks verify that the random controls are not winning by sampling an easier or incomparable space. In particular, D-Rand draws only typed DAGs that satisfy the same feasibility, depth, branch-count, and operator-typing constraints as the guided DAG variants. Flat-Rand is intentionally simpler and is therefore interpreted as a separate stress-test baseline; the equal-space random-control conclusion rests primarily on D-Rand.

3.5 Datasets, Budgets, and Metrics

We evaluate the configuration matrix on the 56 OpenML tabular classification tasks used by DeepLine [14]. We reuse this task collection for two reasons. First, it anchors PrepBench to a prior data-preparation pipeline-search benchmark rather than to a newly curated task set. Second, it provides a fixed public task collection for evaluating preprocessing-pipeline search under controlled budgets. We do not filter datasets by the observed performance of any PrepBench configuration.

Table 4: Benchmark scope after applying the common completeness rule.

Quantity	Value
Initial task collection	56
Datasets analyzed	56
Configurations	9
Seeds per dataset	5
Budgets	60s, 300s, 600s
Runs per budget	2,520
Primary metric	Balanced accuracy
Aggregation unit	Dataset

To keep the statistical block complete for all configurations, seeds, and budgets, we apply a common completeness rule before analysis: a dataset is included only if every configuration–seed–budget cell has a valid run summary under the shared trace schema. All 56 tasks satisfy this rule and are included in the main analysis (Table 4). The rule is applied once, before any configuration comparison, so all methods are evaluated on the same dataset set. The full task list and basic dataset statistics are reported in the supplementary material.

Each configuration is run with five seeds, {42, 114, 256, 512, 768}, and three wall-clock budgets, 60s, 300s, and 600s. The primary quality metric is balanced accuracy. For cross-dataset inference, we first average scores over seeds within each dataset–configuration cell and then treat the dataset as the unit of generalization. We report mean balanced accuracy, 95% confidence intervals, average rank, dataset wins, full-evaluation counts, screened candidates, pruned candidates, and paired budget deltas. All experiments were run on two AMD EPYC 7543 32-core processors and 512 GB of RAM.

4 STATISTICAL PROTOCOL AND MEASUREMENT

A benchmark’s contribution rests on its measurement protocol. We make the protocol explicit and adopt the standard methodology for comparing methods over multiple datasets [7], consistent with the statistical discipline used in data-cleaning-for-ML benchmarks [20].

Aggregation unit. Scores are averaged over the five seeds within each (dataset, configuration) cell before any cross-dataset statistic is computed. The dataset is the unit of analysis, giving a complete 56×9 block per budget.

Why marginal confidence intervals are not enough. Balanced accuracy varies more across datasets than across configurations, so marginal confidence intervals on configuration means cannot by themselves establish or rule out method differences. We report them for transparency but base comparative claims on paired and rank-based tests that control for dataset difficulty.

Omnibus and post-hoc tests. For each budget, we run a Friedman test across the nine configurations on the per-dataset scores. When it rejects the null of equal performance, we apply the Nemenyi post-hoc test. With $k = 9$ configurations, $N = 56$ datasets, and $q_{0.05} = 3.102$, the critical difference is $CD = 1.605$.

Targeted pairwise tests. For specific claims, we use Wilcoxon signed-rank tests [30] on paired per-dataset scores and control the

family-wise error rate within each budget with Holm–Bonferroni correction [15].

Statistical vs. practical significance. With hundreds of paired observations, small systematic shifts can be statistically detectable but not practically meaningful. We therefore report effect magnitude—median delta, fraction of runs improved, and accuracy gained per additional full evaluation—alongside p -values.

Equivalence testing. A non-significant difference is not evidence of equality. To make the “no accuracy change” claims positive rather than merely null, we use two one-sided tests (TOST) on the paired per-dataset differences. The main text uses an equivalence margin of $\delta = 0.01$ balanced accuracy, which corresponds to one percentage point and is smaller than the typical cross-dataset variation in this benchmark. We also report a sensitivity analysis for $\delta \in \{0.005, 0.01, 0.02\}$ to show whether the equivalence verdicts are margin-dependent. Two configurations are declared statistically equivalent only if the procedure rejects both one-sided hypotheses at $\alpha = 0.05$, i.e., the mean difference is confidently inside $[-\delta, +\delta]$.

4.1 Trace Semantics and Score Provenance

The artifact records both run-level summaries and candidate-level histories. The run summary stores the final reported score, score source, number of completed full evaluations, number of staged candidates screened out, and number of budget-pruned candidates. The candidate log stores one row per candidate event, including the run id, candidate id, workflow id, candidate status, stage score, full score when available, elapsed evaluation time, estimated serial and critical-path costs, and whether the candidate updated the incumbent. A separate workflow table maps each workflow id to a canonical signature: structure type, operator sequence or DAG encoding, branch operators, tail operators, depth, branch count, estimator, and hyperparameter choices.

The provenance audit checks two properties. First, whenever a run completes at least one full evaluation, the reported score must be recoverable from a scored candidate row in the trace. Second, the run-level allocation counters must agree with the candidate-level statuses after aggregation.

The audit reveals one important short-budget case: some 60s runs report a positive score even though no full candidate evaluation completed. These scores come from a fallback/default path rather than from a logged full candidate. This case is not treated as a search success. Instead, the revised trace schema records fallback evaluations explicitly using

`status` $\in \{\text{ok}, \text{screened_out}, \text{pruned}, \text{timeout}, \text{failed}, \text{fallback}\}$

and adds a `score_source` field with values such as `full_candidate`, `stage1`, and `fallback`. Under this schema, a reported score has an explicit source and the run-level allocation counters can be audited against the candidate trace when candidate histories are present. In the final 300s/600s runs, the run-level matrices are complete (2,520/2,520 runs at each budget), while the candidate histories cover 2,492/2,520 runs at 300s and 2,510/2,520 runs at 600s. We therefore use the complete run summaries for score-based statistical tests and restrict trace-only diagnostics to runs with candidate histories.

This audit is part of the benchmark contribution. It prevents a misleading interpretation of tight-budget runs: at 60s, a method can

Table 5: Verdict of each configuration against the DAG baseline D-S-F-A, from Holm-corrected Wilcoxon tests (direction) and TOST equivalence ($\delta = 0.01$). ▲ Better, ≈ Equivalent, ? Inconclusive (neither significant nor equivalence-confirmed); no configuration is significantly worse.

Configuration	60s	300s	600s
Flat-Rand (random)	?	▲	▲
D-Rand (random)	?	▲	▲
L-S-F-A (linear)	?	?	?
B-S-F-A (branch)	?	?	?
D-C-F-A (critical-path)	≈	≈	≈
D-S-F-B (budget-aware)	≈	≈	≈
D-S-T-A (staged)	≈	≈	≈
D-C-T-B (combined)	≈	≈	≈

appear to have a positive score without having actually completed a full pipeline evaluation. For this reason, the 60s results are reported as a tight-budget feasibility diagnostic, while the main accuracy conclusions emphasize the 300s and 600s budgets and are repeated on the completed-evaluation-only block.

F1 (Provenance). The run-level matrices are complete for the main 300s and 600s budgets, and the candidate-level histories cover 98.9% and 99.6% of those runs, respectively. Score-based conclusions use the complete run summaries; trace-only allocation and provenance diagnostics are reported on the covered trace subset and clearly separated from the main score tests.

5 EXPERIMENTAL FINDINGS

Table 5 is a roadmap: for every configuration and budget it gives the verdict against the plain DAG baseline D-S-F-A, derived from the Holm-corrected Wilcoxon tests and the TOST equivalence tests below. The pattern is stark, the only configurations that are significantly *better* than the baseline are the two random controls, and every cost-aware structured variant is statistically *equivalent* to it. The rest of this section substantiates and explains these verdicts, organized by research question.

5.1 RQ1: Search-Space Structure and Random Feasible Controls

The strongest configurations in final balanced accuracy are not the most structured ones. Table 6 summarizes the matrix, and Table 7 reports the omnibus tests. The Friedman test rejects the null of equal performance at every budget ($\chi^2 = 34.4$, $p = 3.4 \times 10^{-5}$ at 60s; $\chi^2 = 77.8$, $p = 1.4 \times 10^{-13}$ at 300s; $\chi^2 = 94.4$, $p = 6.0 \times 10^{-17}$ at 600s), so post-hoc comparisons are warranted.

Figure 1 shows the critical-difference diagram at 600s. The two random controls (Flat-Rand, rank 2.85; D-Rand, rank 3.00) form one statistically indistinguishable group, and the seven structured and fixed configurations form another (ranks 5.34–6.20). The gap between the groups exceeds the critical difference of 1.605, so the random controls are significantly better in rank than every guided or hand-designed configuration. The pairwise tests in Table 8 agree: both random controls significantly beat the plain DAG baseline

Table 6: Controlled-matrix summary on the complete 56-dataset benchmark. Scores are mean balanced accuracy over datasets after averaging seeds (best per column in bold). Rank is average rank across datasets; lower is better. Wins counts datasets won (ties split). The critical-difference diagram in Fig. 1 is generated from the same rank values. Marginal 95% CIs on the means are $\approx \pm 0.055$ – 0.063 (see Section 4); comparative inference uses Tables 7 and 8.

Config	Mean score			Avg. rank			Dataset wins		
	60s	300s	600s	60s	300s	600s	60s	300s	600s
Flat-Rand	0.7203	0.7534	0.7595	3.77	2.88	2.85	19.3	21.2	18.2
D-Rand	0.7221	0.7477	0.7564	3.96	3.33	3.00	13.8	16.7	18.7
L-S-F-A	0.7141	0.7386	0.7428	5.58	5.57	5.57	5.8	4.7	5.7
B-S-F-A	0.6669	0.7277	0.7398	5.67	5.91	6.20	5.3	2.8	1.8
D-S-F-A	0.7168	0.7340	0.7405	4.77	5.49	5.53	2.4	0.8	2.3
D-C-F-A	0.7209	0.7343	0.7404	5.06	5.31	5.65	1.7	4.3	0.3
D-S-F-B	0.7204	0.7346	0.7405	4.99	5.43	5.38	3.4	1.8	3.3
D-S-T-A	0.7140	0.7348	0.7407	5.65	5.38	5.48	1.6	2.3	3.3
D-C-T-B	0.7161	0.7344	0.7415	5.55	5.70	5.34	2.6	1.3	2.3

Table 7: Friedman omnibus test across the nine configurations and Nemenyi critical difference (CD) per budget. $k=9$, $N=56$, $\alpha=0.05$.

Budget	Friedman χ^2	p -value	CD
60s	34.4	3.4×10^{-5}	1.605
300s	77.8	1.4×10^{-13}	1.605
600s	94.4	6.0×10^{-17}	1.605

Table 8: Targeted pairwise comparisons (Wilcoxon signed-rank on paired per-dataset scores, Holm-adjusted within budget). “A vs. B” tests whether A differs from B; ns marks $p \geq 0.05$.

Comparison	p (60s)	p (300s)	p (600s)
<i>Structured vs. plain DAG baseline D-S-F-A</i>			
D-C-F-A vs. D-S-F-A	0.54 ns	1.00 ns	1.00 ns
D-S-F-B vs. D-S-F-A	0.54 ns	0.69 ns	1.00 ns
D-S-T-A vs. D-S-F-A	0.12 ns	1.00 ns	1.00 ns
D-C-T-B vs. D-S-F-A	0.50 ns	1.00 ns	1.00 ns
<i>Random controls vs. structured</i>			
Flat-Rand vs. D-S-F-A	0.54 ns	3.1×10^{-4}	3.6×10^{-5}
D-Rand vs. D-S-F-A	0.18 ns	3.1×10^{-4}	8.8×10^{-6}
Flat-Rand vs. D-C-T-B	0.54 ns	2.9×10^{-4}	7.3×10^{-5}
D-Rand vs. D-C-T-B	0.12 ns	7.7×10^{-5}	2.6×10^{-5}

D-S-F-A at 300s and 600s (Holm-adjusted $p \leq 3.1 \times 10^{-4}$). The one nuance is budget-dependent: at 60s the random controls are numerically strong but not significant after Holm correction, so we treat 60s as a tight-budget diagnostic rather than the primary attribution setting.

This changes the interpretation of structured data-preparation search. A comparison against only linear or hand-designed baselines could make the DAG variants look competitive. Once random feasible preparation pipelines are included, the apparent advantage of guided structure-aware search disappears. Random controls are therefore not sanity checks here; they are central baselines.

F2 (Structure, within this EA substrate). Within an evolutionary substrate ($P=10$) over a fixed operator library and a 60–600 s budget range, preparation structure does not buy accuracy: random feasible pipelines match or outrank every guided and hand-designed variant (rank gap $>$ CD at 300 s and 600 s) using only 38–48% of the baseline’s full evaluations. Structure should therefore always be validated against random feasible controls.

5.2 Zero-Penalty Sensitivity: Structure Is Not Suppressed by the EA Objective

A natural alternative explanation for RQ1 is that the EA survival objective could suppress richer structures: it includes small penalties for estimated cost and graph complexity even though the reported incumbent is selected by raw balanced accuracy. We therefore re-run the key DAG comparison with both penalties set to zero, using the same datasets, seeds, operator library, estimator family, and wall-clock budgets. The sensitivity includes D-Rand, the plain guided DAG D-S-F-A, and the combined cost-aware configuration D-C-T-B.

Table 9 shows that the ranking does not reverse. At 300s, D-Rand reaches 0.7478 mean balanced accuracy, compared with 0.7341 for D-S-F-A and 0.7279 for D-C-T-B. At 600s, the corresponding means are 0.7550, 0.7417, and 0.7373. Paired by dataset after averaging the five seeds, D-S-F-A remains below D-Rand at both budgets ($p = 6.15 \times 10^{-5}$ at 300s; $p = 1.79 \times 10^{-5}$ at 600s), and D-C-T-B remains below D-Rand as well ($p = 4.17 \times 10^{-5}$ at 300s; $p = 6.01 \times 10^{-6}$ at 600s). The random-control finding is therefore not explained by the cost/complexity terms in the default EA survival objective.

Trace audit and optimizer strength. The same zero-penalty traces rule out an “easier random space” explanation (Table 10): D-Rand explores simpler pipelines and fewer distinct workflows than the guided DAG variants at comparable per-evaluation runtime. We also increase the guided EA population and offspring size from 10 to 40 under the zero-penalty objective at 300s. This brings the plain guided DAG into the same score range as the random reference (0.7489 vs. 0.7478 mean balanced accuracy), showing that optimizer strength matters; however, D-C-T-B remains below D-S-F-A (0.7427 vs. 0.7489, paired Wilcoxon $p = 7.43 \times 10^{-8}$), so the combined cost-aware variant is still not an accuracy winner.

F3 (Zero-penalty sensitivity). Removing the cost and complexity penalties from the guided EA survival objective does not reverse the main ranking at 300s or 600s. A stronger zero-penalty EA can bring the plain DAG search into the same score range as D-Rand, but the combined cost-aware variant remains below the plain DAG in final balanced accuracy.

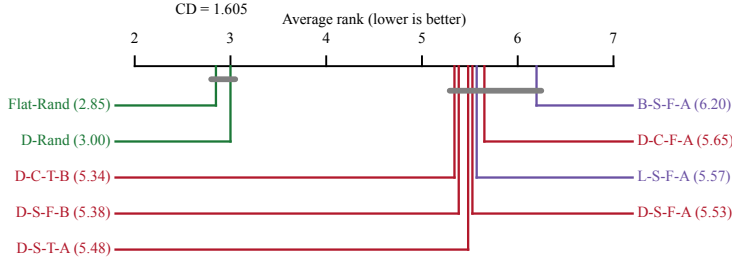


Figure 1: Critical-difference diagram for the nine controlled configurations under the 600s budget (Nemenyi post-hoc test, $\alpha = 0.05$). Lower average rank is better, and configurations connected by a horizontal bar are not significantly different. Flat-Rand and D-Rand form the best-ranked group and are significantly better in rank than the guided structured/cost-aware cluster, illustrating the main RQ1 finding that richer preparation structure does not improve final balanced accuracy within this fixed substrate and budget regime.

Table 9: Zero-penalty sensitivity. Both the estimated-cost penalty and graph-complexity penalty in the EA survival objective are set to zero. Paired tests use per-dataset scores after averaging five seeds.

Budget	Config	Mean	Median	Avg. rank
300s	D-Rand	0.7478	0.7842	1.54
300s	D-S-F-A	0.7341	0.7700	2.05
300s	D-C-T-B	0.7279	0.7720	2.41
600s	D-Rand	0.7550	0.7897	1.52
600s	D-S-F-A	0.7417	0.7722	2.07
600s	D-C-T-B	0.7373	0.7829	2.41

Table 10: Search-space fairness audit (zero-penalty traces, Section 5.2). Pipeline structure and exploration statistics for the equal-space random control D-Rand and the guided DAG baseline D-S-F-A. D-Rand satisfies the same typing, depth, and branch-count constraints, yet explores simpler pipelines and fewer distinct workflows at comparable per-evaluation cost, so its accuracy advantage is not explained by an easier or larger search space.

Budget	Config	Nodes	Branches	Uniq. workflows	t_{eval} (s)
300s	D-Rand	2.40	1.31	1,403	2.86
	D-S-F-A	2.97	1.79	4,867	3.08
600s	D-Rand	2.40	1.31	2,340	2.93
	D-S-F-A	3.26	1.96	8,781	3.24

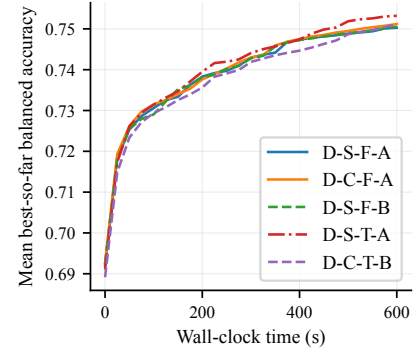


Figure 2: Incumbent convergence of the guided EA variants under the 600s budget. Each curve shows the mean best-so-far balanced accuracy reconstructed from completed full-validation candidates and averaged over dataset-seed runs. All guided variants improve rapidly early in the budget and then plateau, supporting the budget-scaling result in Table 14.

5.3 RQ2: Cost-Aware Policies Change Allocation, Not Accuracy

The combined configuration D-C-T-B uses a DAG representation, a critical-path cost model, staged evaluation, and budget-aware pruning. It substantially changes how the budget is spent (Table 13). At 300s it performs 413.2 full evaluations on average, screens 233.7 candidates, and prunes 6,345.0; the full-evaluation DAG baseline D-S-F-A performs 612.5 full evaluations with no screening or pruning. At 600s the pattern remains: 912.1 full evaluations for D-C-T-B versus 1,368.3 for D-S-F-A.

Despite this large allocation shift, the final-score difference is not statistically significant: the Holm-adjusted Wilcoxon p -value for D-C-T-B vs. D-S-F-A is not significant at any budget (Table 8), with per-dataset mean delta +0.0005 at 300s and +0.0010 at 600s and median delta approximately 0. The same holds for each individual mechanism: adding a critical-path cost model (D-C-F-A), budget-aware pruning (D-S-F-B), or staged evaluation (D-S-T-A) leaves accuracy statistically indistinguishable from D-S-F-A. Staged and budget-aware mechanisms should therefore be evaluated as resource-allocation mechanisms, not as accuracy improvements.

The non-significant differences are not merely a failure to detect an effect. Equivalence testing makes the claim positive: under TOST with a one-percentage -point margin ($\delta = 0.01$), all four structured variants are *statistically equivalent* to the DAG baseline at both 300s and 600s (Table 12). The accuracy of the structured and cost-aware configurations is thus confidently within 0.01 of the plain DAG, so their value, if any, lies in the allocation profile rather than the outcome.

The equivalence verdicts in Table 12 use a single margin $\delta = 0.01$. Because a TOST verdict can in principle hinge on the chosen margin, we re-run the equivalence tests at $\delta \in \{0.005, 0.01, 0.02\}$ on the same paired per-dataset differences; Table 11 reports the outcome at 300 s and 600 s. The four structured variants (D-C-F-A,

D-S-F-B, D-S-T-A, D-C-T-B) are confirmed equivalent to the plain DAG baseline D-S-F-A at every margin $\delta \geq 0.01$ at both budgets. At the tight margin $\delta = 0.005$, the cost-model and pruning variants (D-C-F-A, D-S-F-B) remain equivalent, whereas the staged variants (D-S-T-A, D-C-T-B) become *inconclusive*: their mean differences (+0.0008 and +0.0005 at 300 s) sit inside a half-percentage-point band but the one-sided tests no longer both reject at $\alpha = 0.05$. Crucially, no configuration flips to *significantly different* at any margin, and at $\delta = 0.005$ a half-percentage-point effect is already below this benchmark’s cross-dataset noise (marginal 95% CIs of ± 0.055 – 0.063 , Section IV). The allocation-not-accuracy reading of RQ2 is therefore not an artifact of the $\delta = 0.01$ choice: at any practically meaningful margin the cost-aware machinery is statistically equivalent to the plain DAG, and the worst case at an implausibly tight margin is “no detectable difference,” never “better.”

Table 11: Equivalence-margin sensitivity. TOST verdict of each structured variant against the DAG baseline D-S-F-A at three margins δ . $\approx$ statistically equivalent ($p < \alpha = 0.05$ on both one-sided tests); “?” inconclusive (neither equivalence-confirmed nor significant). No variant is significantly different at any margin.

Config	300 s			600 s		
	$\delta=.005$	.01	.02	$\delta=.005$	.01	.02
D-C-F-A	$\approx$	$\approx$	$\approx$	$\approx$	$\approx$	$\approx$
D-S-F-B	$\approx$	$\approx$	$\approx$	$\approx$	$\approx$	$\approx$
D-S-T-A	?	$\approx$	$\approx$	?	$\approx$	$\approx$
D-C-T-B	?	$\approx$	$\approx$	?	$\approx$	$\approx$

F4 (Cost-aware policies). Critical-path costing, staged evaluation, and budget-aware pruning are resource-allocation mechanisms, not accuracy improvements: they are statistically *equivalent* to the plain DAG within one accuracy point while shifting full-evaluation counts by roughly one third.

5.4 RQ3: Budget Scaling and Mechanism Diagnostics

Table 14 reports paired budget deltas on exactly matched (dataset, configuration, seed) runs. Moving from 60s to 300s yields gains that are both statistically significant and practically meaningful: every configuration improves with Wilcoxon $p < 10^{-23}$, the largest gain is B-S-F-A (+0.0607 mean), and the random controls improve by $0.0256 - 0.0331$ on average.

From 300 s to 600 s the picture inverts into a statistical/practical split. The Wilcoxon test still rejects the null for every configuration ($p < 0.05$ across configurations) because many small positive increments accumulate over matched runs, but the median delta is 0 for every configuration and only 29.6–42.5% of runs improve. For the plain DAG baseline D-S-F-A, 755.7 additional full evaluations buy a mean gain of only +0.0065. The reliable conclusion is a plateau after 300s: extra budget is statistically detectable, but it does not change the rank-level conclusion or rescue the guided structured variants.

Table 12: Equivalence to the DAG baseline D-S-F-A (TOST on paired per-dataset differences, margin $\delta = 0.01$). The four cost-aware DAG variants are statistically equivalent to the baseline ($p < \alpha = 0.05$).

Config	300s		600s	
	mean Δ	TOST p	mean Δ	TOST p
D-C-F-A	+0.0003	1.2×10^{-26}	−0.0001	9.3×10^{-32}
D-S-F-B	+0.0006	7.6×10^{-28}	+0.0000	2.3×10^{-31}
D-S-T-A	+0.0008	1.4×10^{-13}	+0.0002	7.5×10^{-16}
D-C-T-B	+0.0005	4.2×10^{-14}	+0.0010	2.6×10^{-11}

Table 13: Evaluation allocation (mean count per run) at 300s and 600s. Full evaluations are full-validation evaluations (Section 4.1).

Config	300s			600s		
	Full	Screened	Pruned	Full	Screened	Pruned
Flat-Rand	293.3	0.0	0.0	607.6	0.0	0.0
D-Rand	254.1	0.0	0.0	518.7	0.0	0.0
L-S-F-A	683.3	0.0	0.0	1560.7	0.0	0.0
B-S-F-A	468.9	0.0	0.0	1086.4	0.0	0.0
D-S-F-A	612.5	0.0	0.0	1368.3	0.0	0.0
D-C-F-A	613.8	0.0	0.0	1461.7	0.0	0.0
D-S-F-B	647.8	0.0	4464.3	1459.3	0.0	5449.2
D-S-T-A	434.0	249.2	0.0	953.6	614.5	0.0
D-C-T-B	413.2	233.7	6345.0	912.1	532.9	5450.2

Incumbent convergence. To check whether the guided variants fail simply because they do not have enough time to converge, we reconstruct incumbent trajectories from the candidate-level trace. For each run, we record the best completed full-validation score observed up to each elapsed-time point and average the resulting best-so-far curves over dataset–seed runs. Fig. 2 shows the 600s budget. All guided structured variants improve rapidly early in the budget and then flatten, consistent with the small 300s-to-600s gains in Table 14. The convergence view therefore supports the budget-scaling result: additional evaluations after the early search phase rarely change the final rank-level conclusion. The random controls are evaluated separately through final balanced accuracy, full-evaluation counts, and the efficiency frontier in Table 6, Table 13, and Fig. 5.

F5 (Budget). Most budget value is realized by 300 s: the 60 s $\rightarrow$ 300 s gain is significant and meaningful (all $p < 10^{-23}$), whereas the 300 s $\rightarrow$ 600 s gain, though still detectable (all $p < 0.05$), has median 0 for every configuration and leaves the random-control dominance intact. Longer budgets are therefore not a substitute for a better search policy.

Table 14: Paired budget deltas on matched (dataset, configuration, seed) runs. Δ score is the mean change with its 95% CI; the median, the fraction of runs improved, the mean extra full evaluations, and the Wilcoxon p -value are also shown. Note the contrast between the two budget pairs.

Budget pair	Config	Mean Δ (95% CI)	Median Δ	Improved	Extra full evals	Wilcoxon p
300s – 60s	Flat-Rand	+0.0331 (± 0.0108)	+0.0030	0.614	239.1	1.9×10^{-29}
	D-Rand	+0.0256 (± 0.0088)	+0.0040	0.664	204.8	4.1×10^{-32}
	L-S-F-A	+0.0245 (± 0.0093)	+0.0021	0.607	601.0	1.0×10^{-27}
	B-S-F-A	+0.0607 (± 0.0228)	+0.0035	0.629	407.4	2.9×10^{-28}
	D-S-F-A	+0.0172 (± 0.0089)	+0.0017	0.614	531.4	4.5×10^{-24}
	D-C-F-A	+0.0134 (± 0.0049)	+0.0029	0.625	529.8	1.1×10^{-26}
	D-S-F-B	+0.0142 (± 0.0052)	+0.0025	0.614	560.7	3.0×10^{-25}
	D-S-T-A	+0.0208 (± 0.0076)	+0.0030	0.664	379.9	9.0×10^{-30}
600s – 300s	D-C-T-B	+0.0183 (± 0.0069)	+0.0025	0.643	358.5	3.0×10^{-28}
	Flat-Rand	+0.0061 (± 0.0041)	+0.0000	0.325	314.2	7.0×10^{-15}
	D-Rand	+0.0088 (± 0.0052)	+0.0000	0.350	264.6	2.3×10^{-16}
	L-S-F-A	+0.0042 (± 0.0015)	+0.0000	0.332	877.4	2.1×10^{-14}
	B-S-F-A	+0.0121 (± 0.0095)	+0.0000	0.296	617.5	2.6×10^{-13}
	D-S-F-A	+0.0065 (± 0.0042)	+0.0000	0.368	755.7	4.1×10^{-15}
	D-C-F-A	+0.0061 (± 0.0042)	+0.0000	0.350	847.9	3.4×10^{-12}
	D-S-F-B	+0.0060 (± 0.0041)	+0.0000	0.379	811.5	1.0×10^{-13}
	D-S-T-A	+0.0059 (± 0.0041)	+0.0000	0.396	519.7	4.3×10^{-14}
	D-C-T-B	+0.0071 (± 0.0041)	+0.0000	0.425	499.0	4.6×10^{-17}

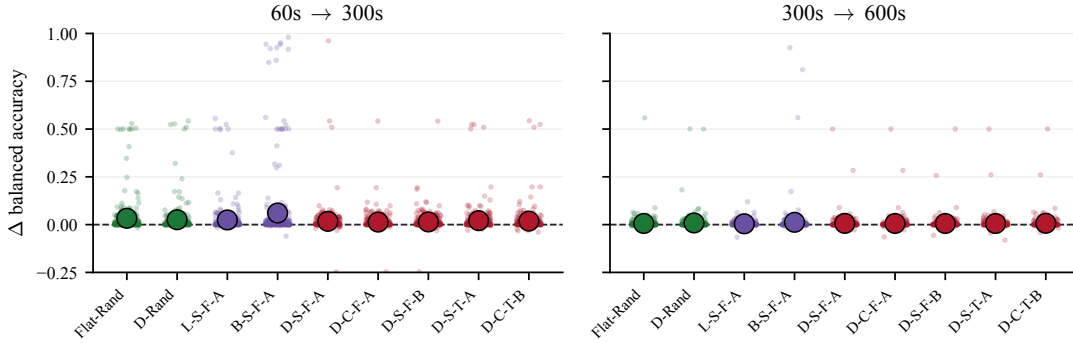


Figure 3: Paired budget deltas. Each small point is one matched (dataset, seed) run; large circles show the per-configuration mean. Moving from 60 s to 300 s gives sizable score gains; moving from 300 s to 600 s adds many evaluations for negligible gain, clustering near zero.

5.5 Mechanism: Staged Screening Is Conservative and Accuracy-Neutral

Staged evaluation screens many candidates, but its aggregate accuracy effect is indistinguishable from zero. Comparing D-S-T-A with D-S-F-A, the Holm-adjusted Wilcoxon p -value is 1.00 at both the 300 s and 600 s budgets (and non-significant at 60 s), with median delta 0, even though D-S-T-A screens hundreds of candidates and performs roughly 30% fewer full evaluations (Table 13). The combined configuration behaves the same way.

The candidate-level audit explains why staging saves evaluations without moving accuracy: its screening is well-calibrated and conservative (Table 15, Fig. 4). The stage-1 subsample score predicts the full-validation score strongly (Pearson ≈ 0.95 , Spearman 0.93 – 0.95

at all budgets); the promotion rate is 0.60–0.65 (about a third of candidates are screened); and the incumbent-threat rate is 0.0, meaning that no screened candidate had a stage-1 score above its run’s eventual full-validation incumbent. This is not a true false-negative rate because screened candidates are not normally full-evaluated. It is a trace diagnostic showing that staging removes candidates that appear unlikely to threaten the incumbent, which reduces full evaluations while leaving the final balanced accuracy unchanged. This trace explains why staging changes allocation without changing final balanced accuracy.

F6 (Staged screening). Staged screening is calibrated and conservative (stage-1/full $r \approx 0.95$, zero incumbent-threat cases), so it saves evaluations without changing final accuracy.

Table 15: Staged-screening diagnostics from the candidate-level trace. Stage-1 score predicts the full score well among promoted candidates, and about a third of candidates are screened. The last column is the incumbent-threat rate: the fraction of screened candidates whose stage-1 score exceeded the run’s eventual full-validation incumbent. It is not a true false-negative rate because screened candidates are not normally full-evaluated.

Config	Budget	Pearson	Spearman	Promotion	Threat
D-S-T-A	60s	0.954	0.941	0.648	0.000
D-S-T-A	300s	0.948	0.936	0.635	0.000
D-S-T-A	600s	0.951	0.945	0.608	0.000
D-C-T-B	60s	0.953	0.943	0.650	0.000
D-C-T-B	300s	0.947	0.940	0.639	0.000
D-C-T-B	600s	0.952	0.945	0.631	0.000

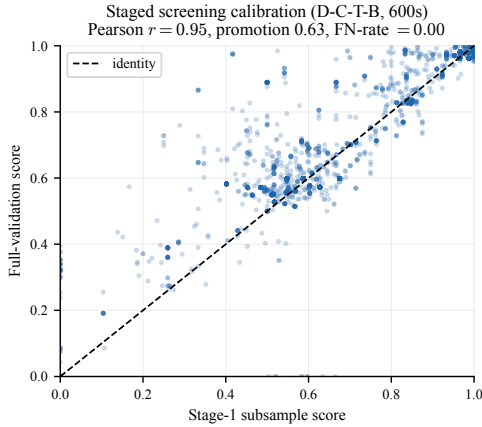


Figure 4: Stage-1 subsample score vs. full-validation score for promoted candidates (D-C-T-B, 600s; dashed line is identity). The strong correlation, together with a zero incumbent-threat rate, shows that screening removes candidates that appear unlikely to win—explaining why it saves full evaluations without changing final balanced accuracy.

5.6 Mechanism: Random Controls Are on the Efficiency Frontier

Because the structured configurations are statistically equivalent in accuracy (Table 12), the meaningful axis among them is how many evaluations they spend. Fig. 5 plots mean balanced accuracy against mean full evaluations per run at 600s. The two random controls occupy the upper-left—the best accuracy for the fewest full evaluations—while every structured configuration lies lower and to the right. Relative to the DAG baseline D-S-F-A (1,368.3 full evaluations per run), Flat-Rand and D-Rand reach higher accuracy with only 44% and 38% of the full evaluations, respectively. Among the structured configurations, staging and pruning do improve evaluation economy without hurting accuracy: D-C-T-B uses 67% of the baseline’s full evaluations (Table 13), but it also incurs 532.9

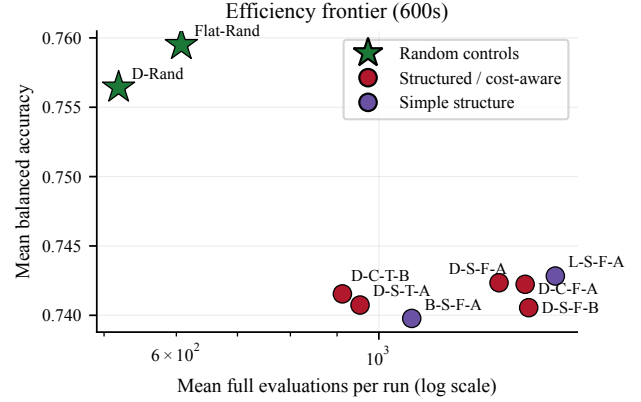


Figure 5: Accuracy vs. full evaluations per run at 600s. The random controls (stars) dominate the structured configurations: higher accuracy with fewer full evaluations. Structured and cost-aware configurations trade evaluation count against each other at statistically equivalent accuracy.

screened and 5,450.2 pruned candidates per run, so its total candidate throughput is high even though its full-evaluation count is low. The efficiency ranking therefore depends on whether the budgeted resource is full evaluations or total candidate work—a distinction a single accuracy number cannot express, and one that benchmarks of preparation-pipeline search should report explicitly.

F7 (Efficiency frontier). Random controls alone occupy the accuracy–evaluation frontier; staging and pruning save full evaluations but raise total candidate work.

5.7 Mechanism: The Cost Model Is a Coarse Signal

The critical-path and budget-aware policies act on a static cost estimate, so its fidelity matters. Two measurements characterize it. First, on the same DAGs the critical-path estimate is 7–11% lower than the serial-sum estimate (Table 16), confirming that the cost-model axis is active rather than nominal; for the serial configurations the two estimates coincide by construction. Second, the static estimate is only weakly predictive of realized per-evaluation runtime: pooled across completed runs, the coefficient of determination between the cost estimate and the measured per-evaluation runtime is $R^2 \leq 0.04$ at every budget (Pearson correlation $\lesssim 0.2$; Table 16). A model that explains under 5% of the runtime variance is a coarse guide, which helps explain why budget-aware pruning reshuffles allocation but does not translate into better outcomes: pruning decisions rest on estimates that are loosely coupled to the costs that actually consume the budget.

F8 (Cost model). The static estimate that drives pruning explains under 5% of realized per-evaluation runtime ($R^2 \leq 0.04$), so cost-aware pruning reshuffles allocation on a coarse signal, calibrating or learning the cost model is the concrete lever for making it pay off.

Table 16: Cost-model calibration. Critical-path reduction is the percentage by which the critical-path estimate falls below the serial-sum estimate on the same plans; R^2 is for the static estimate predicting realized per-evaluation runtime (completed runs, pooled).

Budget	D-C-F-A red.	D-C-T-B red.	cost→runtime R^2
60s	7.4%	7.1%	0.01
300s	8.6%	10.4%	0.03
600s	8.9%	11.1%	0.04

This result turns a negative accuracy finding into an actionable systems diagnosis: future preparation-search systems should either calibrate the static cost model on observed runtime, learn a dataset- and operator-aware runtime model, or report that their pruning decisions are based on an uncalibrated estimate.

5.8 RQ4a: The Conclusions Survive Removing Fallback Runs

To confirm that the fallback runs of Section 4.1 do not drive the results, we repeat the analysis on the completed-evaluation-only block (Section 4). The clean block retains all 56 datasets at every budget. The Friedman test still rejects equality ($p = 7.5 \times 10^{-10}$, 2.5×10^{-14} , and 4.2×10^{-20} at 60s/300s/600s), the random controls keep the two best average ranks at 300s and 600s, and the structured variants remain non-significant against D-S-F-A. At 60s, where fallback runs are most common, D-Rand becomes significantly better than the baseline on the clean block ($p \approx 10^{-4}$). Thus the headline conclusions are not an artifact of the fallback path.

5.9 Implications for Benchmarking Preparation-Pipeline Search

The matrix yields a diagnostic conclusion rather than a state-of-the-art claim, with three implications. First, data-preparation-search studies should include random feasible preparation controls; without them it is easy to attribute performance to structure-aware search when the same space is explored effectively by random sampling under short and medium budgets. Second, cost-aware and staged mechanisms should be evaluated with both quality and systems metrics: D-C-T-B is not an accuracy winner, but it offers a different allocation profile (fewer full evaluations, many pruned candidates) that may be desirable when full evaluations are costly. Third, longer budgets are not a substitute for calibrated search: the 60s→300s move helps, but the 300s→600s move yields only negligible median gains, motivating trace-level analysis of why guided search fails to convert extra evaluations into better incumbents.

5.10 RQ4b: External End-to-End Reference: CtxPipe

The controlled matrix isolates design axes inside one substrate, but it does not by itself show how the score range compares with a recent end-to-end preparation-search system. We therefore report CtxPipe [12] as an external reference, separated from the factorized attribution matrix because it uses its own policy, operator choices,

implementation, and time-accounting assumptions. Scored with the same balanced-accuracy metric, five seeds, and downstream split on the 56 datasets, the public CtxPipe reference obtains mean 0.7045 and median 0.7380. The controlled random references are higher in mean (0.7477–0.7564 for D-Rand, 0.7534–0.7595 for Flat-Rand) and win on 40–47 of 56 paired datasets; the per-dataset-best controlled oracle wins on 44–48 datasets. Since the public CtxPipe reference does not expose the same wall-clock budget interface, we use it only to contextualize score ranges, not to attribute gains.

6 LIMITATIONS AND THREATS TO VALIDITY

PrepBench isolates design axes inside one execution substrate, so its negative accuracy result should not be read as a universal claim that structured preparation search cannot help. Different operators, estimators, or optimizers may exploit the space differently; indeed, increasing the guided EA population and offspring size brings the plain DAG into the random-control score range at 300s. The result is nevertheless not explained by the default estimated-cost or graph-complexity penalties, since the zero-penalty sensitivity preserves the main ranking at 300s and 600s. The workload is also scoped to 56 public OpenML tabular classification tasks and balanced accuracy; regression, text, time series, streaming cleaning, calibration, fairness, latency, memory, and human repair cost may favor other policies. Finally, wall-clock budgets and external systems depend on hardware and implementation choices, so we report evaluation counts and trace diagnostics, keep CtxPipe as contextual rather than attributional evidence, and treat staged-screening results as an incumbent-threat diagnostic rather than a true false-negative rate over all screened candidates.

7 REPRODUCIBILITY AND ARTIFACT

The released artifact provides run-level summaries for every (dataset, configuration, seed, budget) cell, candidate-level histories for the covered runs, workflow identifiers, and scripts that regenerate the tables and figures. Additional task lists and dataset statistics are provided in the supplementary material.

8 CONCLUSION

PrepBench factorizes cost-aware data-preparation pipeline search into preparation structure, execution-cost model, evaluation policy, and budget policy, and evaluates these axes in one controlled substrate. The benchmark’s main lesson is methodological. Within the tested tabular workloads and 60–600s budget regime, random feasible preparation pipelines are strong baselines: they match or out-rank the guided structured variants, and critical-path costing, staged evaluation, and budget-aware pruning are statistically equivalent to a plain DAG baseline in final balanced accuracy. Zero-penalty and stronger-optimizer sensitivities refine but do not overturn this result: optimizer strength matters, while the combined cost-aware variant’s observed value lies mainly in allocation, screening, pruning, and evaluation economy. Preparation-search studies should therefore report random feasible controls, paired multi-dataset tests, equivalence tests, candidate provenance, and cost-model calibration. PrepBench provides the reusable benchmark and artifact for running those checks.

REFERENCES

- [1] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *The journal of machine learning research* 13, 1 (2012), 281–305.
- [2] Laure Berti-Equille. 2019. Learn2Clean: Optimizing the Sequence of Tasks for Web Data Preparation. In *The World Wide Web Conference* (San Francisco, CA, USA) (WWW '19). Association for Computing Machinery, New York, NY, USA, 2580–2586. <https://doi.org/10.1145/3308558.3313602>
- [3] Felix Biessmann, Tammo Rukat, Philipp Schmidt, Prathik Naidu, Sebastian Schelter, Andrey Taptunov, Dustin Lange, and David Salinas. 2019. DataWig: Missing value imputation for tables. *Journal of Machine Learning Research* 20, 175 (2019), 1–6.
- [4] Sibe Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. <https://doi.org/10.1145/3588945>
- [5] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*. 2201–2206.
- [6] Tamraparni Dasu and Theodore Johnson. 2003. *Exploratory data mining and data cleaning*. John Wiley & Sons.
- [7] Janez Demšar. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *J. Mach. Learn. Res.* 7 (Dec. 2006), 1–30.
- [8] Xiaoou Ding, Hongzhi Wang, Jiaxuan Su, Zijue Li, Jianzhong Li, and Hong Gao. 2019. Cleanits: a data cleaning system for industrial time series. *Proceedings of the VLDB Endowment* 12, 12 (2019), 1786–1789.
- [9] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. 2020. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. [arXiv:2003.06505 \[stat.ML\]](https://arxiv.org/abs/2003.06505) <https://arxiv.org/abs/2003.06505>
- [10] Matthias Feurer, Katharina Eggensperger, Stefan Falkner, Marius Lindauer, and Frank Hutter. 2022. Auto-sklearn 2.0: hands-free AutoML via meta-learning. *J. Mach. Learn. Res.* 23, 1, Article 261 (Jan. 2022), 61 pages.
- [11] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2* (Montreal, Canada) (NIPS'15). MIT Press, Cambridge, MA, USA, 2755–2763.
- [12] Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6, Article 231 (Dec. 2024), 27 pages. <https://doi.org/10.1145/3698831>
- [13] Pieter Gijsbers and Joaquin Vanschoren. 2019. GAMA: Genetic Automated Machine Learning Assistant. *Journal of Open Source Software* (2019).
- [14] Yuval Heffetz, Roman Vainshtein, Gilad Katz, and Lior Rokach. 2020. DeepLine: AutoML Tool for Pipelines Generation using Deep Reinforcement Learning and Hierarchical Actions Filtering. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) (KDD '20). Association for Computing Machinery, New York, NY, USA, 2103–2113. <https://doi.org/10.1145/3394486.3403261>
- [15] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian journal of statistics* (1979), 65–70.
- [16] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning: Methods, Systems, Challenges* (1st ed.). Springer Publishing Company, Incorporated.
- [17] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: interactive data cleaning for statistical modeling. *Proc. VLDB Endow.* 9, 12 (Aug. 2016), 948–959. <https://doi.org/10.14778/2994509.2994514>
- [18] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2017. Hyperband: a novel bandit-based approach to hyperparameter optimization. *J. Mach. Learn. Res.* 18, 1 (Jan. 2017), 6765–6816.
- [19] Peng Li, Zhiyi Chen, Xu Chu, and Kexin Rong. 2023. DiffPrep: Differentiable Data Preprocessing Pipeline Search for Learning over Tabular Data. *Proc. ACM Manag. Data* 1, 2, Article 183 (June 2023), 26 pages. <https://doi.org/10.1145/3589328>
- [20] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 13–24. <https://doi.org/10.1109/ICDE51399.2021.00009>
- [21] Felix Mohr, Marcel Wever, and Eyke Hüllermeier. 2018. ML-Plan: Automated machine learning via hierarchical planning. *Mach. Learn.* 107, 8–10 (Sept. 2018), 1495–1515. <https://doi.org/10.1007/s10994-018-5735-z>
- [22] Avani Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 738–746. <https://doi.org/10.14778/3574245.3574258>
- [23] Randal S. Olson, Nathan Bartley, Ryan J. Urbanowicz, and Jason H. Moore. 2016. Evaluation of a Tree-based Pipeline Optimization Tool for Automating Data Science. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016* (Denver, Colorado, USA) (GECCO '16). Association for Computing Machinery, New York, NY, USA, 485–492. <https://doi.org/10.1145/2908812.2908918>
- [24] Shafaq Siddiqi, Roman Kern, and Matthias Boehm. 2023. SAGA: A Scalable Framework for Optimizing Data Cleaning Pipelines for Machine Learning Applications. *Proc. ACM Manag. Data* 1, 3, Article 218 (Nov. 2023), 26 pages. <https://doi.org/10.1145/3617338>
- [25] Henan Sun, Xunkai Li, Daohan Su, Junyi Han, Rong-Hua Li, and Guoren Wang. 2024. Towards Data-centric Machine Learning on Directed Graphs: a Survey. [arXiv:2412.01849 \[cs.LG\]](https://arxiv.org/abs/2412.01849) <https://arxiv.org/abs/2412.01849>
- [26] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2013. Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Chicago, Illinois, USA) (KDD '13). Association for Computing Machinery, New York, NY, USA, 847–855. <https://doi.org/10.1145/2487575.2487629>
- [27] Haluk Topcuoglu, Salim Hariri, and Min-you Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Trans. Parallel Distrib. Syst.* 13, 3 (March 2002), 260–274. <https://doi.org/10.1109/71.993206>
- [28] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2014. OpenML: networked science in machine learning. *SIGKDD Explor. Newsl.* 15, 2 (June 2014), 49–60. <https://doi.org/10.1145/2641190.2641198>
- [29] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. 2021. FLAML: A Fast and Lightweight AutoML Library. In *Proceedings of Machine Learning and Systems*.
- [30] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 196–202.
- [31] Alice Zheng and Amanda Casari. 2018. *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O'Reilly Media.
- [32] Donald W Zimmerman and Bruno D Zumbo. 1993. Relative power of the Wilcoxon test, the Friedman test, and repeated-measures ANOVA on ranks. *The Journal of Experimental Education* 62, 1 (1993), 75–86.